

Project 1 - Manual Order Registry

Compact engineering summary: contracts, ownership, lifecycle, and the raw-pointer problems deliberately exposed.

Goal

Move from CP-style code in **main()** to a small component with a responsibility, public API, explicit ownership model, invariants, implementation file, and a testable lifecycle.

What was built

Order is a value type containing **id**, **price**, **quantity**, and **side**. It owns no resource, so the compiler-generated copy constructor is correct. **OrderRegistry** uses **std::map<OrderId, Order*>** and exposes **create**, **find**, **cancel**, and a destructor. Copy construction and copy assignment are deleted.

Ownership model

```
caller creates Order
-> create(const Order&) borrows caller object temporarily
-> registry creates new Order(order) on the heap
-> registry alone owns that heap Order

find(id)    -> borrowed Order* or nullptr
cancel(id)  -> registry deletes owned Order, then erases map entry
destructor  -> deletes every still-owned Order
```

Contracts

```
create: ID must be absent; assert enforces that programmer contract
find:  absence is normal -> nullptr; a non-null pointer is borrowed only
cancel: ID must be active; registry calls delete; prior borrows become dangling
copying: forbidden; this is a unique-owner registry
```

Lifecycle proof

A smoke test verified: an unknown lookup returns **nullptr**; **create()** stores a copy at a different address from the caller object; **find()** returns expected field values; **cancel()** removes the entry; a later lookup returns **nullptr**. At scope exit, the registry destructor runs automatically.

What this project taught

Header/source split: headers declare the usable contract; **.cpp** files define the algorithm. **Copy construction:** **new Order(order)** creates a new object and invokes the compiler-generated copy constructor. **Borrowing:** **const Order&** provides temporary access without ownership transfer. **Assertions:** document/enforce programmer preconditions during development. **RAII:** object destruction triggers registry cleanup.

Raw-pointer failure modes

```
store &order          -> caller object can leave scope -> dangling pointer
caller deletes find() result -> registry later deletes again -> double delete / UB
erase only            -> heap Order leaks
delete only           -> map retains dangling pointer
default-copy registry -> two registries own same addresses -> double delete
duplicate create unchecked -> overwrites old pointer -> old Order leaks
map destructor alone   -> map nodes die; raw pointees do not
```

Important distinction: map copy is not resource deep copy

Copying **std::map<OrderId, Order*>** creates independent map nodes but copies pointer address values. Both maps then point to the same heap Orders. A true deep registry copy would allocate a separate **new Order(*old_order)** for every source entry, then carefully clean up if copying fails halfway.

Deliberately deferred work

Exception safety: if **new Order(order)** succeeds but map insertion throws, manual raw-pointer code can leak. **Const-correct observation:** a mutable **Order*** lets callers change **Order::id** and violate the map-key invariant. **Production errors:** assertions are not ordinary error reporting and are usually disabled in release builds. Future RAII with **std::unique_ptr**, clear error results, and stronger observer APIs solve these classes of problems.

Bottom line

Manual ownership is not "just use new/delete." Every pointer creates a lifetime contract: who owns it, who may observe it, who deletes it, what invalidates it, and what copying means. This project made those obligations visible before modern C++ tools automate them.